

This plan is designed to be executed within a 24-48 hour hackathon timeframe, focusing on achieving the core functionality of the "Cascading API Mesh Failure" scenario.

Part 1: E-commerce Platform Setup (The Target System)

The platform will be built as a multi-service application deployed using Kubernetes (K8s). We will use Docker Compose locally and deploy the final manifests to a K8s cluster (like Minikube or a cloud cluster).

A. Core Services (Microservices)

Service	Role	Key Dependency	Simulated Fault Injection Point	Metric Focus (OpenTelemetry)
S1: Checkout Service	Entry point, orchestrates the order.	S2 (Inventory)	High Latency Spike (Inject 500ms delay).	Request Latency, Error Rate.
S2: Inventory Service	Manages stock levels.	S3 (Pricing)	Connection Pool Saturation (Simulate slow DB calls).	DB Query Duration, Active Connections.
S3: Pricing Service	Calculates final product price.	None	Garbage Collection Pauses (Inject GC load/time spikes).	GC Pause Duration, Memory Utilization.
S4: Payment Gateway Proxy	Handles external payment calls.	S2 (Inventory)	None (Acts as a stressed client).	Queue Length, Success Rate.
S5: Recommendation Service	Suggests related products.	S2 (Inventory)	None (Acts as a stressed client).	Request Timeout Count.

B. Data Generation & Observability Stack

- Application Instrumentation:** All services (S1-S5) are written (e.g., in Go or Python) and instrumented using OpenTelemetry. They emit metrics (latency, CPU, memory) and traces (request flow) to the OpenTelemetry Collector.
- Telemetry Ingestion:** The OTel Collector aggregates data and pushes it into **Prometheus**.
- Visualization:** **Grafana** is set up to pull data from Prometheus, displaying dashboards for each service and, critically, a **dependency graph** showing the real-time state (latency, errors) of all services.
- Alerting:** Prometheus AlertManager is configured to fire critical alerts based on the scenario:
 - Primary Alert:** `S1_Request_Latency_p99 > 500ms` (Triggers the AIRP).

Part 2: AIRP Deployment (The Brain)

The AIRP will be deployed as a collection of coordinated services running on the same K8s cluster.

A. Core Infrastructure Components

- Workflow Orchestration Engine (Temporal):** Acts as the central state machine. When S1 alerts, Prometheus calls a Temporal workflow, which dictates the entire incident resolution flow (Detect $\rightarrow$ Correlate $\rightarrow$ Analyze $\rightarrow$ Wait $\rightarrow$ Execute $\rightarrow$ Validate).
- AI Agents (LangGraph/CrewAI):** These are the business logic units, deployed as dedicated microservices (e.g., Python FastAPI endpoints). They are called by the Temporal workflow at specific stages.
 - Agents:* Monitoring Agent, Correlation Agent, RCA Agent, Remediation Agent, Documentation Agent.
- Database/Knowledge Base: PostgreSQL + Vector Extension** stores:
 - Historical Incidents (for RCA context).
 - Service Metadata (Dependency graph definition).
 - Operational Runbook/KB (Textual knowledge for GPT to search).

B. Agent Logic Implementation

Agent	Technology	Primary Function	Inputs (from Temporal)	Outputs (to Temporal)
Monitoring Agent	Go/Python	Initial Triage; verifies alert validity.	<code>Alert_Payload</code> (S1 Latency Spike).	<code>Validated_Incident_Object</code> (Confirmed time window, severity).
Correlation Agent	Python (Go)	Maps symptoms to underlying causes.	<code>Validated_Incident_Object</code> .	<code>Correlated_Incident_Object</code> (Impacted list, Primary Strain, Candidate Cause).
RCA Agent	LangGraph/GPT	Deep reasoning using KB/History.	<code>Correlated_Incident_Object</code> .	<code>Diagnosis_Report</code> (Probable Cause, Confidence Score).
Remediation Agent	Python (LangChain/GPT)	Generates actionable plan.	<code>Diagnosis_Report</code> .	<code>Remediation_Plan</code> (Recommended Action, Rollback Path).
Documentation Agent	Python (GPT)	Compiles final narrative.	<code>Validated_Incident_Object</code> + <code>Remediation_Plan</code> + Validation Status.	<code>Incident_Report_Markdown</code> (Final RCA).



Part 3: The Complete Data Flow & Sequence

This flow is executed by the Temporal Workflow Engine.

[START] $\rightarrow$ 1. **Alert Firing (Prometheus):** `S1_Latency > 500ms` .
 $\downarrow$
2. **Workflow Start (Temporal):** Temporal receives the alert and initiates the `IncidentResolutionWorkflow` .
 $\downarrow$
3. **Monitoring Agent:** Runs validation query on OTel/Prometheus. $\rightarrow$ **Validates Incident.**
 $\downarrow$
4. **Correlation Agent:** Queries Service Metadata. $\rightarrow$ **Identifies S2/S3/S5 as related.**
 $\downarrow$
5. **RCA Agent:** Queries KB/History via GPT. $\rightarrow$ **Diagnoses S3 GC Pause (88% confidence).**
 $\downarrow$
6. **Remediation Agent:** Queries Runbooks/KB via GPT. $\rightarrow$ **Recommends "Scale S3 by 2."**
 $\downarrow$
7. **Governance Checkpoint (Temporal Logic):** Checks policy (Scale $\uparrow$ is OK). **Pulls Human Approval.**
 $\downarrow$
8. **Human Action (SRE Team):** Approves the recommendation.
 $\downarrow$
9. **Execution Layer (K8s Operator):** Temporal calls a Golang service that executes `kubectl scale S3` .
 $\downarrow$
10. **Validation Loop (Monitoring Agent):** Monitors S1, S2 latency/pool metrics. **Checks for recovery (300ms/70% utilization).**
 $\downarrow$
11. **Recovery Confirmed?** (Yes/No)
* **IF YES:** $\rightarrow$ 12. **Documentation Agent:** Compiles the final report.
* **IF NO:** $\rightarrow$ Back to Step 6 (Remediation Agent) to try a different plan (e.g., "Restart S3").
 $\downarrow$
13. **[END]** Final report is logged to PostgreSQL and pushed to Grafana/Slack.

 Part 4: Hackathon Execution Plan (Timeline Estimate)

Phase	Focus	Tasks	Technologies	Estimated Time	Goal/Deliverable
Sprint 1: Infrastructure & Target System (6 hrs)	Get the platform running and producing data.	1. Spin up K8s/Minikube. 2. Deploy S1-S5 services. 3. Instrument S1-S5 with OpenTelemetry. 4. Configure Prometheus/Grafana/Alerting. 5. Implement Fault Injection: Ensure the "S3 GC" and "S1 Latency" can be turned on/off.	K8s, Docker, OpenTelemetry, Prometheus, Grafana, Go/Python.	6 hours	Working E-commerce Platform with a critical, measurable alert.

Phase	Focus	Tasks	Technologies	Estimated Time	Goal/Deliverable
Sprint 2: Workflow Core & Triage (8 hrs)	Build the state machine and initial detection logic.	1. Set up Temporal cluster. 2. Implement the <code>IncidentResolutionWorkflow</code> . 3. Deploy Monitoring Agent and hook it to the workflow start. 4. Implement Correlation Agent (basic dependency lookup). 5. Wire the flow: Alert $\rightarrow$ Monitor $\rightarrow$ Correlate.	Temporal, Golang/Python, Prometheus API, PostgreSQL.	8 hours	The system successfully receives an alert and identifies all related services (S1, S2, S3, S5).
Sprint 3: AI Reasoning & Remediation (8 hrs)	Add intelligence and decision-making.	1. Deploy RCA Agent (LangGraph). 2. Implement Vector Search on the KB (Runbooks/History). 3. Test GPT prompt for Root Cause Diagnosis (S3 $\rightarrow$ GC). 4. Deploy Remediation Agent and implement the rollback logic. 5. Implement Governance Checkpoint (Human approval stub).	LangGraph, GPT API, Vector DB (pg_vector), PostgreSQL.	8 hours	The system diagnoses the root cause and suggests a validated, safe action.
Sprint 4: Execution & Polish (4 hrs)	Complete the loop and finalize the experience.	1. Implement K8s Operator interaction (Golang service for <code>kubect1</code>). 2. Wire the Execution layer to the workflow. 3. Implement Validation Loop (Monitor post-fix). 4. Deploy Documentation Agent and integrate final report generation. 5. Demo Polish: Create a slick Grafana dashboard showing the incident lifecycle.	Temporal, Golang, K8s, GPT API.	4 hours	Fully Autonomous Loop. The platform resolves the Cascading Failure end-to-end.